

Omega Operator Command Manual

Repository: `screeps_omega`

Repository version used: `410d1ce52281b2502403c299c02bee91e1d4aedc`

Generation date: `2026-06-17 18:18:01 PDT`

Source authority: `src/main.js`, `src/ops.js`, `src/market_console.js`,
`src/room_reporting.js`, `src/pcl_manager.js`, `src/power_manager.js`,
`src/empire_manager.js`, `src/reservation_manager.js`, `src/attack_manager.js`,
`src/ops_logistics_manager.js`, and `src/terminal_balance_manager.js`.

This manual documents commands discovered from the source tree. Existing documentation was not used as command authority.

Authoritative operator-facing command surface generated from source.

1. Getting Started

Omega exposes its operator surface through runtime globals registered every tick by `src/main.js`: `ops` and `market`. `src/ops.js` also registers global `view(mode)` plus convenience constants `on` and `off`. `src/market_console.js` registers `market`.

Command philosophy:

- Use `ops` for empire, room, logistics, reservation, attack, power, Power Creep, observer, CPU, and reporting workflows.
- Use `market` for market visibility, terminal staging, dry-run planning, market execution preflight, explicit market execution, and history.
- Most commands print operator-readable console output and return either that printed string, a lines array, or a structured result object.
- Commands that mutate memory are still operator commands: they create plans, set policies, stage logistics requests, cancel requests, or update console workflow memory.

Approval-gated workflow:

- Power Creep actions use a `check` versus `confirm` pattern. `check` reports readiness and does not call the Screeps API action. `confirm` is required before commands call `powerCreep.usePower(...)` or `powerCreep.enableRoom(...)`.
- Market plan execution is a two-step workflow: create or inspect a saved plan, run `market.executionDryRun(planId)`, then intentionally run `market.executePlan(planId)`.
- Direct manual market commands `market.buy(...)`, `market.sell(...)`, and `market.send(...)` execute immediately after local validation. They are not dry-run commands.

Execution levels used in this manual:

- Read Only: reads runtime or Memory state and prints a report.
- Planning: writes Memory, creates a plan, creates a logistics request, changes policy, or clears stored workflow/log history without directly calling market deal, terminal send, or Power Creep APIs.
- Approval Required: requires an explicit `confirm` or saved-plan execution command before calling an executing Screeps API.
- Executes Immediately: directly calls an execution API such as `Game.market.deal`, `terminal.send`, `powerCreep.usePower`, or `powerCreep.enableRoom`.

2. Global Command Index

Global aliases and constants

- `view(on|off)`
- `on`
- `off`

Ops commands

- `ops.attack(targetRoom, [postAction], [parentRoom], [allies])`
- `ops.attacks()`
- `ops.balanceTerminal(roomName)`
- `ops.balanceTerminals()`
- `ops.cancel(requestId)`
- `ops.cancelAttack(targetRoom)`
- `ops.cancelExpansion(targetRoom)`

- ops.cancelReserve(targetRoom)
- ops.clearTerminal(roomName, [resource], [amount])
- ops.cpu([roomName])
- ops.cpuStatus(roomName)
- ops.empire()
- ops.expand(targetRoom, [parentRoom])
- ops.expansions()
- ops.fillTerminal(roomName, resource, amount)
- ops.help()
- ops.hud(on|off)
- ops.log([roomName], [limit])
- ops.logClear([roomName])
- ops.move(resource, amount, roomName, from, to)
- ops.operator(name, [roomName], ["powers"])
- ops.operator(name, roomName, "operateExtension", mode)
- ops.operator(name, roomName, "operateSpawn", [spawnNameOrId], mode)
- ops.ops([roomName], ["stage", from, to, amount])
- ops.pcl([roomName])
- ops.phase(roomName)
- ops.power([roomName], [mode], [on|off])
- ops.powerCreep(name, "assign", room)
- ops.powerCreep(name, "generateOps", mode)
- ops.powerCreep(name, "renewAssist", on|off)
- ops.powerCreep(name, "renewStatus")
- ops.powerCreep(name, action, room, [target], [mode])
- ops.powerCreeps()
- ops.powerEnable(roomName, mode, [name])
- ops.reports(on|off)
- ops.reserve(targetRoom, [parentRoom])
- ops.reserved([parentRoom])
- ops.room([roomName], [section])
- ops.rooms()
- ops.requests([roomName], ["all"|"history"])
- ops.terminalStatus([roomName])
- ops.tickRate([sampleTicks|status|cancel])
- ops.tickSpeed([sampleTicks|status|cancel])

Market commands

- market.buy(resource, amount, roomName)
- market.buyOptions([resource])
- market.cancel(requestId)
- market.clearExecutionLimit(name)

- `market.clearHistory(mode)`
- `market.clearLimit(name)`
- `market.clearPlan(planId)`
- `market.clearPlan("all")`
- `market.clearPlans()`
- `market.deletePlan(planId)`
- `market.execute(planId)`
- `market.executePlan(planId)`
- `market.executionDryRun(planId)`
- `market.executionLimits()`
- `market.executionStatus()`
- `market.help()`
- `market.history([resource|roomName|"all"])`
- `market.historyAudit()`
- `market.historyLimit()`
- `market.historySummary()`
- `market.info()`
- `market.install()`
- `market.limits()`
- `market.needs()`
- `market.opportunities([resource])`
- `market.ping()`
- `market.plan(planId)`
- `market.planAudit()`
- `market.planBuy(resource, amount, roomName)`
- `market.planBuys()`
- `market.planReview(planId)`
- `market.planSell(resource, amount, roomName)`
- `market.planSells()`
- `market.planSummary()`
- `market.plans(["all"|"history"])`
- `market.readiness([roomName|resource])`
- `market.recommendations()`
- `market.removePlan(planId)`
- `market.requests([roomName], ["all"|"history"])`
- `market.restore()`
- `market.rooms()`
- `market.sell(resource, amount, roomName)`
- `market.sellOptions([resource])`
- `market.send(resource, amount, fromRoom, toRoom)`
- `market.setExecutionLimit(name, value)`
- `market.setHistoryLimit(limit)`
- `market.setLimit(name, value)`

- `market.stage(resource, amount, roomName)`
- `market.stock([roomName])`
- `market.surplus()`
- `market.uninstall()`
- `market.unstage(resource, amount, roomName)`

3. Command Reference

Global

`view(mode)`

Description: Toggle HUD and critical reports together and optionally print the current room report when enabling.

Parameters: mode: boolean, number, or string toggle.

Example: `view(on)`

Returns: Object with enabled, HUD, and report state.

Side Effects: Updates ops view, HUD, and report flags.

Execution Level: Planning

Source: `src/ops.js:1037`, `src/ops.js:1987`

`on`

Description: Convenience global value for `true`.

Parameters: None.

Example: `ops.hud(on)`

Returns: Boolean constant.

Side Effects: None.

Execution Level: Read Only

Source: `src/ops.js:920`

`off`

Description: Convenience global value for `false`.

Parameters: None.

Example: `ops.reports(off)`

Returns: Boolean constant.

Side Effects: None.

Execution Level: Read Only

Source: `src/ops.js:921`

Ops

`ops.help()`

Description: Print the ops help surface.

Parameters: None.

Example: `ops.help()`

Returns: Help row array.

Side Effects: Prints help.

Execution Level: Read Only

Source: `src/ops.js:930`, `src/ops.js:1042`

`ops.hud(mode)`

Description: Toggle the room HUD overlay.

Parameters: mode: on, off, boolean, or equivalent.

Example: `ops.hud(on)`

Returns: Toggle result object.

Side Effects: Updates HUD flag in ops state.

Execution Level: Planning

Source: `src/ops.js:924`, `src/ops.js:1050`

`ops.reports(mode)`

Description: Toggle critical room reports.

Parameters: mode: on, off, boolean, or equivalent.

Example: `ops.reports(off)`

Returns: Toggle result object.

Side Effects: Updates reports flag in ops state.

Execution Level: Planning

Source: `src/ops.js:927`, `src/ops.js:1064`

`ops.room(arg1, arg2)`

Description: Show one room report. If one argument is a section, uses the current/default room.

Parameters: arg1: room name or section. arg2: section. Sections: overview, economy, build, defense, creeps, sources, advanced, power, observer, cpu, resources, all.

Example: `ops.room("W5N5", "power")`

Returns: Report object, except CPU returns a printable string.

Side Effects: Sets current room and may update room progress.

Execution Level: Read Only

Source: `src/ops.js:933`, `src/ops.js:1078`, `src/room_reporting.js:1396`

`ops.cpu(roomName)`

Description: Show measured room CPU, top section costs, pressure, and scheduler skips.

Parameters: roomName: optional owned room.

Example: `ops.cpu("W5N5")`

Returns: Printable CPU report status string.

Side Effects: Sets current room through `ops.room`.

Execution Level: Read Only

Source: `src/ops.js:936`, `src/ops.js:1983`

`ops.cpuStatus(roomName)`

Description: Alias for `ops.room(roomName, "cpu")`.

Parameters: `roomName`: optional owned room.

Example: `ops.cpuStatus("W5N5")`

Returns: Printable CPU report status string.

Side Effects: Sets current room through `ops.room`.

Execution Level: Read Only

Source: `src/ops.js:1029`, `src/ops.js:1979`

`ops.phase(roomName)`

Description: Alias for the build section of `ops.room`.

Parameters: `roomName`: optional owned room.

Example: `ops.phase("W5N5")`

Returns: Room report object for build section.

Side Effects: Sets current room through `ops.room`.

Execution Level: Read Only

Source: `src/ops.js:1032`, `src/ops.js:1987`

`ops.rooms()`

Description: Show overview lines for all owned rooms.

Parameters: None.

Example: `ops.rooms()`

Returns: Room reports array.

Side Effects: Updates progress in generated reports.

Execution Level: Read Only

Source: `src/ops.js:939`, `src/ops.js:1713`

`ops.empire()`

Description: Show empire summary and owned-room overview.

Parameters: None.

Example: `ops.empire()`

Returns: Empire report object.

Side Effects: Updates progress in generated reports.

Execution Level: Read Only

Source: `src/ops.js:942`, `src/ops.js:1728`

`ops.log(arg1, arg2)`

Description: Show compact invasion history.

Parameters: `arg1`: room name or limit. `arg2`: optional limit.

Example: `ops.log("W43N6")`

Returns: Lines array.

Side Effects: Prints invasion log lines.

Execution Level: Read Only

Source: `src/ops.js:945`, `src/ops.js:1745`

`ops.logClear(roomName)`

Description: Clear invasion history for one room, or all rooms when omitted.

Parameters: roomName: optional room or all.

Example: `ops.logClear("W43N6")`

Returns: Clear result object.

Side Effects: Deletes invasion log entries.

Execution Level: Planning

Source: `src/ops.js:948`, `src/ops.js:1772`

`ops.tickRate(sampleTicks)`

Description: Sample wall-clock milliseconds per tick over a short window; supports status and cancel.

Parameters: sampleTicks: positive integer, status, or cancel.

Example: `ops.tickRate(5)`

Returns: Printable status line.

Side Effects: Stores or clears probe state under runtime ops console memory.

Execution Level: Planning

Source: `src/ops.js:951`, `src/ops.js:1787`

`ops.tickSpeed(sampleTicks)`

Description: Alias for `ops.tickRate`.

Parameters: Same as `ops.tickRate`.

Example: `ops.tickSpeed("status")`

Returns: Printable status line.

Side Effects: Same as `ops.tickRate`.

Execution Level: Planning

Source: `src/ops.js:954`

`ops.power(roomName, arg1, arg2)`

Description: Show empire Power Spawn status, show a room power report, or set room-local processing/refill policy.

Parameters: roomName: optional room. arg1: detail, on, off, process, refill, or reserve amount. arg2: toggle or amount depending on mode.

Example: `ops.power("W5N5", "process", "off")`

Returns: Printable report or policy line.

Side Effects: Policy forms update room power policy memory.

Execution Level: Planning

Source: `src/ops.js:957`, `src/ops.js:1107`, `src/power_manager.js:242`

`ops.pcl(roomName)`

Description: Show GPL/PCL status and optional room enablement readiness.

Parameters: roomName: optional room.

Example: `ops.pcl("W5N5")`

Returns: Printable report string.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/ops.js:960`, `src/ops.js:1155`, `src/pcl_manager.js:2265`

`ops.powerCreeps()`

Description: List friendly Power Creeps without controlling them.

Parameters: None.

Example: `ops.powerCreeps()`

Returns: Printable report string.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/ops.js:963`, `src/ops.js:1161`, `src/pcl_manager.js:2289`

`ops.operator(powerCreepName, roomName, mode, targetOrMode, maybeMode)`

Description: Report operator readiness or check/confirm manual OPERATE_SPAWN and OPERATE_EXTENSION.

Parameters: `powerCreepName`: name. `roomName`: room. `mode`: powers, operateSpawn, or operateExtension. Additional target/mode args as needed.

Example: `ops.operator("OperatorOne", "W5N5", "operateSpawn", "Spawn1", "check")`

Returns: Printable report string.

Side Effects: confirm for operate commands calls `powerCreep.usePower`; report mode does not.

Execution Level: Approval Required

Source: `src/ops.js:966`, `src/ops.js:1167`, `src/pcl_manager.js:2379`

`ops.powerCreep(powerCreepName, action, roomName, targetOrMode, mode)`

Description: Assign, unassign, report, check, or confirm Power Creep lifecycle and movement actions.

Parameters: `action`: assign, unassign, renewAssist, renewStatus, spawn, renew, position, move, generateOps.

Example: `ops.powerCreep("OperatorOne", "move", "W5N5", "powerSpawn", "check")`

Returns: Printable line or report string.

Side Effects: Assignment and renew-assist update memory. confirm lifecycle/generateOps calls Power Creep API actions.

Execution Level: Approval Required

Source: `src/ops.js:969`, `src/ops.js:1226`, `src/pcl_manager.js:2407`

`ops.ops(roomName, action, from, to, amount, powerCreepName)`

Description: Show ops resource inventory or stage RESOURCE_OPS between room storage and terminal.

Parameters: `roomName`: optional room. `action`: optional stage. `from/to`: storage or terminal. `amount`: positive number.

Example: `ops.ops("W5N5", "stage", "storage", "terminal", 1000)`

Returns: Printable inventory block or request line.

Side Effects: Stage creates an ops logistics move request.

Execution Level: Planning

Source: `src/ops.js:972`, `src/ops.js:1331`

ops.powerEnable(roomName, mode, powerCreepName)

Description: Check room power enablement readiness or confirm enableRoom.

Parameters: roomName: owned room. mode: check or confirm. powerCreepName: required when confirming if more than one candidate exists.

Example: ops.powerEnable("W5N5", "check")

Returns: Printable report string.

Side Effects: confirm calls powerCreep.enableRoom.

Execution Level: Approval Required

Source: src/ops.js:975, src/ops.js:1409, src/pcl_manager.js:2578

ops.move(resource, amount, roomName, from, to)

Description: Create a room-local logistics request between storage and terminal.

Parameters: resource: resource constant or string. amount: positive number. roomName: room. from/to: endpoints.

Example: ops.move("H", 50000, "W42N9", "terminal", "storage")

Returns: Logistics result object.

Side Effects: Creates a move request in ops logistics memory.

Execution Level: Planning

Source: src/ops.js:978, src/ops.js:1451

ops.terminalStatus(roomName)

Description: Show terminal capacity, energy, resources, and congestion status.

Parameters: roomName: optional owned room.

Example: ops.terminalStatus("W42N9")

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: src/ops.js:981, src/ops.js:1460

ops.clearTerminal(roomName, resource, amount)

Description: Create terminal-to-storage logistics requests for terminal cleanup.

Parameters: roomName: owned room. Optional resource and amount; when omitted, source selects cleanup resources.

Example: ops.clearTerminal("W42N9", "H", 50000)

Returns: Result object or cleanup summary.

Side Effects: Creates one or more logistics requests.

Execution Level: Planning

Source: src/ops.js:984, src/ops.js:1518

ops.fillTerminal(roomName, resource, amount)

Description: Create a storage-to-terminal logistics request for market staging.

Parameters: roomName, resource, amount.

Example: ops.fillTerminal("W42N9", "energy", 10000)

Returns: Logistics result object.

Side Effects: Creates a logistics request.

Execution Level: Planning

Source: src/ops.js:987, src/ops.js:1684

`ops.requests(roomName, mode)`

Description: Show active or historical ops logistics requests.

Parameters: roomName: optional room. mode: optional all or history.

Example: `ops.requests("W42N9", "all")`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/ops.js:990`, `src/ops.js:1692`

`ops.cancel(requestId)`

Description: Cancel an ops logistics request.

Parameters: requestId: request id.

Example: `ops.cancel("ol_123_W42N9_H_1")`

Returns: Cancel result object.

Side Effects: Marks request canceled.

Execution Level: Planning

Source: `src/ops.js:993`, `src/ops.js:1735`

`ops.balanceTerminal(roomName)`

Description: Evaluate one terminal balance target set and create conservative logistics requests.

Parameters: roomName: owned room.

Example: `ops.balanceTerminal("W42N9")`

Returns: Terminal balance result object.

Side Effects: May create logistics requests.

Execution Level: Planning

Source: `src/ops.js:996`, `src/ops.js:1741`

`ops.balanceTerminals()`

Description: Evaluate terminal balance targets for all owned rooms with storage and terminal.

Parameters: None.

Example: `ops.balanceTerminals()`

Returns: Aggregate result object.

Side Effects: May create logistics requests.

Execution Level: Planning

Source: `src/ops.js:999`, `src/ops.js:1760`

`ops.expand(targetRoom, parentRoom)`

Description: Start or update a manual expansion plan.

Parameters: targetRoom: room to claim. parentRoom: optional support room.

Example: `ops.expand("W5N6", "W5N5")`

Returns: Expansion result object.

Side Effects: Creates/updates expansion memory; may convert reservation to expansion.

Execution Level: Planning

Source: `src/ops.js:1002`, `src/ops.js:1809`, `src/empire_manager.js:1390`

`ops.reserve(targetRoom, parentRoom)`

Description: Start or update a reserved-room plan.

Parameters: targetRoom: remote room. parentRoom: optional parent; current room is used when omitted.

Example: `ops.reserve("W5N6", "W5N5")`

Returns: Reservation result object.

Side Effects: Creates/updates reservation memory; may convert expansion to reservation.

Execution Level: Planning

Source: `src/ops.js:1005`, `src/ops.js:1836`, `src/reservation_manager.js:806`

`ops.reserved(parentRoom)`

Description: Show active reserved rooms grouped by parent.

Parameters: parentRoom: optional parent filter.

Example: `ops.reserved("W5N5")`

Returns: Lines array.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/ops.js:1008`, `src/ops.js:1874`

`ops.expansions()`

Description: Show active expansion plans.

Parameters: None.

Example: `ops.expansions()`

Returns: Lines array.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/ops.js:1011`, `src/ops.js:1880`

`ops.attack(targetRoom, postActionOrOptions, parentRoom, allies)`

Description: Start or update a manual attack plan.

Parameters: targetRoom: target. postAction: expand, reserve, or none; options object also accepted. parentRoom: support room. allies: array.

Example: `ops.attack("W5N6", "expand", "W5N5", ["W4N6"])`

Returns: Attack result object.

Side Effects: Creates/updates attack memory.

Execution Level: Planning

Source: `src/ops.js:1014`, `src/ops.js:1886`, `src/attack_manager.js:301`

`ops.attacks()`

Description: Show active attack plans.

Parameters: None.

Example: `ops.attacks()`

Returns: Lines array.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/ops.js:1017`, `src/ops.js:1936`

`ops.cancelAttack(targetRoom)`

Description: Cancel an active attack plan.

Parameters: `targetRoom`: target room.

Example: `ops.cancelAttack("W5N6")`

Returns: Cancel result object.

Side Effects: Cancels attack plan memory.

Execution Level: Planning

Source: `src/ops.js:1020`, `src/ops.js:1943`, `src/attack_manager.js:395`

`ops.cancelExpansion(targetRoom)`

Description: Cancel an active expansion plan.

Parameters: `targetRoom`: target room.

Example: `ops.cancelExpansion("W5N6")`

Returns: Cancel result object.

Side Effects: Cancels expansion plan memory.

Execution Level: Planning

Source: `src/ops.js:1023`, `src/ops.js:1949`, `src/empire_manager.js:1516`

`ops.cancelReserve(targetRoom)`

Description: Cancel an active reserved-room plan.

Parameters: `targetRoom`: reserved room.

Example: `ops.cancelReserve("W5N6")`

Returns: Cancel result object.

Side Effects: Cancels reservation plan memory.

Execution Level: Planning

Source: `src/ops.js:1026`, `src/ops.js:1955`, `src/reservation_manager.js:871`

Market

`market.help()`

Description: Print the market command help surface.

Parameters: None.

Example: `market.help()`

Returns: Printable block.

Side Effects: Prints help.

Execution Level: Read Only

Source: `src/market_console.js:2135`, `src/market_console.js:3079`

`market.info()`

Description: Show helper runtime and memory status.

Parameters: None.

Example: `market.info()`

Returns: Object with runtime status and memory.

Side Effects: Touches market console memory registration fields.

Execution Level: Planning

Source: `src/market_console.js:2225, src/market_console.js:3080`

`market.ping()`

Description: Confirm market helper availability.

Parameters: None.

Example: `market.ping()`

Returns: Printable pong line.

Side Effects: Prints line.

Execution Level: Read Only

Source: `src/market_console.js:2249, src/market_console.js:3081`

`market.install()`

Description: Install and register market helper in runtime memory.

Parameters: None.

Example: `market.install()`

Returns: Printable line.

Side Effects: Updates market memory and registers global.

Execution Level: Planning

Source: `src/market_console.js:2253, src/market_console.js:3082`

`market.restore()`

Description: Restore a previously uninstalled helper.

Parameters: None.

Example: `market.restore()`

Returns: Printable line.

Side Effects: Updates market memory and registers global.

Execution Level: Planning

Source: `src/market_console.js:2263, src/market_console.js:3083`

`market.uninstall()`

Description: Mark helper uninstalled and delete `global.market` for current runtime.

Parameters: None.

Example: `market.uninstall()`

Returns: Printable line.

Side Effects: Updates memory and deletes runtime global.

Execution Level: Planning

Source: `src/market_console.js:2272, src/market_console.js:3084`

`market.rooms()`

Description: List owned rooms and terminal availability.

Parameters: None.

Example: `market.rooms()`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:2284`, `src/market_console.js:3086`

`market.stock(roomName)`

Description: Show empire or room storage/terminal stock.

Parameters: `roomName`: optional owned room.

Example: `market.stock("W42N9")`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:2301`, `src/market_console.js:3087`

`market.needs()`

Description: Show obvious room-level market/logistics needs.

Parameters: None.

Example: `market.needs()`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:2348`, `src/market_console.js:3088`

`market.surplus()`

Description: Show obvious surplus resources.

Parameters: None.

Example: `market.surplus()`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:2400`, `src/market_console.js:3089`

`market.stage(resource, amount, roomName)`

Description: Stage a resource from storage to terminal.

Parameters: `resource`, `amount`, `roomName`.

Example: `market.stage("H", 10000, "W42N9")`

Returns: Printable logistics message.

Side Effects: Creates ops logistics request.

Execution Level: Planning

Source: `src/market_console.js:2452`, `src/market_console.js:3091`

`market.unstage(resource, amount, roomName)`

Description: Move a resource from terminal to storage.

Parameters: resource, amount, roomName.

Example: `market.unstage("H", 10000, "W42N9")`

Returns: Printable logistics message.

Side Effects: Creates ops logistics request.

Execution Level: Planning

Source: `src/market_console.js:2464`, `src/market_console.js:3092`

`market.requests(roomName, mode)`

Description: Show market-compatible logistics requests.

Parameters: roomName: optional. mode: optional all or history.

Example: `market.requests("W42N9", "all")`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:2476`, `src/market_console.js:3093`

`market.cancel(requestId)`

Description: Cancel an ops or market request by id.

Parameters: requestId: request id.

Example: `market.cancel("ol_123_W42N9_H_1")`

Returns: Printable cancel message.

Side Effects: Marks request canceled when found.

Execution Level: Planning

Source: `src/market_console.js:2541`, `src/market_console.js:3094`

`market.send(resource, amount, fromRoom, toRoom)`

Description: Send resources directly between owned terminals.

Parameters: resource, amount, fromRoom, toRoom.

Example: `market.send("energy", 5000, "W42N9", "W41N8")`

Returns: Printable send result.

Side Effects: Calls `terminal.send`.

Execution Level: Executes Immediately

Source: `src/market_console.js:2551`, `src/market_console.js:3095`

`market.buyOptions(resource)`

Description: Show usable buy options from sell orders.

Parameters: resource: optional resource.

Example: `market.buyOptions("energy")`

Returns: Printable block.

Side Effects: Reads market orders.

Execution Level: Read Only

Source: `src/market_console.js:2599`, `src/market_console.js:3097`

`market.sellOptions(resource)`

Description: Show usable sell options against buy orders.

Parameters: resource: optional resource.

Example: `market.sellOptions("H")`

Returns: Printable block.

Side Effects: Reads market orders.

Execution Level: Read Only

Source: `src/market_console.js:2662`, `src/market_console.js:3098`

`market.readiness(input)`

Description: Show market readiness by empire, room, or resource.

Parameters: input: optional room name or resource.

Example: `market.readiness("H")`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:321`, `src/market_console.js:3099`

`market.opportunities(resource)`

Description: Show ready market sell opportunities.

Parameters: resource: optional resource.

Example: `market.opportunities("H")`

Returns: Printable block.

Side Effects: Reads market and terminal state.

Execution Level: Read Only

Source: `src/market_console.js:595`, `src/market_console.js:3100`

`market.recommendations()`

Description: Show conservative next manual market/logistics commands.

Parameters: None.

Example: `market.recommendations()`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:636`, `src/market_console.js:3101`

`market.planSell(resource, amount, roomName)`

Description: Save a dry-run sell plan.

Parameters: resource, amount, roomName.

Example: `market.planSell("H", 10000, "W42N9")`

Returns: Printable plan report.

Side Effects: Writes saved plan memory.

Execution Level: Planning

Source: `src/market_console.js:994`, `src/market_console.js:3102`

`market.planBuy(resource, amount, roomName)`

Description: Save a dry-run buy plan.

Parameters: resource, amount, roomName.

Example: `market.planBuy("energy", 5000, "W41N8")`

Returns: Printable plan report.

Side Effects: Writes saved plan memory.

Execution Level: Planning

Source: `src/market_console.js:1028, src/market_console.js:3103`

`market.plans(mode)`

Description: List saved plans.

Parameters: mode: optional all or history.

Example: `market.plans("all")`

Returns: Printable block.

Side Effects: May mark expired plans during listing.

Execution Level: Planning

Source: `src/market_console.js:1069, src/market_console.js:3104`

`market.plan(planId)`

Description: Recheck and show one saved plan.

Parameters: planId: saved plan id.

Example: `market.plan("ms_223455_H_W42N9")`

Returns: Printable plan report.

Side Effects: May update plan status.

Execution Level: Planning

Source: `src/market_console.js:1222, src/market_console.js:3105`

`market.planSummary()`

Description: Summarize saved plan counts.

Parameters: None.

Example: `market.planSummary()`

Returns: Printable block.

Side Effects: Reads plan memory.

Execution Level: Read Only

Source: `src/market_console.js:1987, src/market_console.js:3106`

`market.planReview(planId)`

Description: Review saved plan readiness before execution.

Parameters: planId: saved plan id.

Example: `market.planReview("ms_223455_H_W42N9")`

Returns: Printable review report.

Side Effects: May update plan status.

Execution Level: Planning

Source: `src/market_console.js:1230, src/market_console.js:3107`

`market.planAudit()`

Description: Audit active saved plans for stale, duplicate, blocked, or missing-order cases.

Parameters: None.

Example: `market.planAudit()`

Returns: Printable block.

Side Effects: May update stale status through rechecks.

Execution Level: Planning

Source: `src/market_console.js:2052`, `src/market_console.js:3108`

`market.clearPlan(planId)`

Description: Soft-delete one plan or active plans.

Parameters: `planId` or `all`.

Example: `market.clearPlan("all")`

Returns: Printable line.

Side Effects: Marks plan(s) deleted.

Execution Level: Planning

Source: `src/market_console.js:1950`, `src/market_console.js:3109`

`market.deletePlan(planId)`

Description: Deprecated alias for `market.clearPlan(planId)`.

Parameters: `planId`: saved plan id.

Example: `market.deletePlan("ms_223455_H_W42N9")`

Returns: Printable deprecation line and clear result.

Side Effects: Marks plan deleted.

Execution Level: Planning

Source: `src/market_console.js:1972`, `src/market_console.js:3110`

`market.removePlan(planId)`

Description: Deprecated alias for `market.clearPlan(planId)`.

Parameters: `planId`: saved plan id.

Example: `market.removePlan("ms_223455_H_W42N9")`

Returns: Printable deprecation line and clear result.

Side Effects: Marks plan deleted.

Execution Level: Planning

Source: `src/market_console.js:1977`, `src/market_console.js:3111`

`market.clearPlans()`

Description: Deprecated alias for `market.clearPlan("all")`.

Parameters: None.

Example: `market.clearPlans()`

Returns: Printable deprecation line and clear result.

Side Effects: Marks active plans deleted.

Execution Level: Planning

Source: `src/market_console.js:1982`, `src/market_console.js:3112`

`market.executionStatus()`

Description: Show execution design and safety status.

Parameters: None.

Example: `market.executionStatus()`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:1262, src/market_console.js:3113`

`market.executionDryRun(planId)`

Description: Recompute execution preflight without executing.

Parameters: `planId`: saved plan id.

Example: `market.executionDryRun("ms_223455_H_W42N9")`

Returns: Printable preflight report.

Side Effects: Reads current market/terminal state.

Execution Level: Read Only

Source: `src/market_console.js:1520, src/market_console.js:3114`

`market.executionLimits()`

Description: Show configured execution limits.

Parameters: None.

Example: `market.executionLimits()`

Returns: Printable block.

Side Effects: Ensures default limit memory exists.

Execution Level: Planning

Source: `src/market_console.js:1276, src/market_console.js:3115`

`market.setExecutionLimit(name, value)`

Description: Set a numeric execution safety limit.

Parameters: `name`: supported limit. `value`: number or null for `maxBuyEffectivePrice`.

Example: `market.setExecutionLimit("maxSellAmount", 5000)`

Returns: Printable line.

Side Effects: Updates execution limit memory.

Execution Level: Planning

Source: `src/market_console.js:1312, src/market_console.js:3116`

`market.clearExecutionLimit(name)`

Description: Reset one execution limit to its default.

Parameters: `name`: supported limit.

Example: `market.clearExecutionLimit("maxSellAmount")`

Returns: Printable line.

Side Effects: Updates execution limit memory.

Execution Level: Planning

Source: `src/market_console.js:1326, src/market_console.js:3117`

`market.limits()`

Description: Alias for `market.executionLimits()`.

Parameters: None.

Example: `market.limits()`

Returns: Printable block.

Side Effects: Ensures default limit memory exists.

Execution Level: Planning

Source: `src/market_console.js:3118`

`market.setLimit(name, value)`

Description: Alias for `market.setExecutionLimit`.

Parameters: `name`, `value`.

Example: `market.setLimit("maxBuyAmount", 5000)`

Returns: Printable line.

Side Effects: Updates execution limit memory.

Execution Level: Planning

Source: `src/market_console.js:3119`

`market.clearLimit(name)`

Description: Alias for `market.clearExecutionLimit`.

Parameters: `name`.

Example: `market.clearLimit("maxBuyAmount")`

Returns: Printable line.

Side Effects: Updates execution limit memory.

Execution Level: Planning

Source: `src/market_console.js:3120`

`market.executePlan(planId)`

Description: Execute a saved market plan after current-state preflight passes.

Parameters: `planId`: saved plan id.

Example: `market.executePlan("ms_223455_H_W42N9")`

Returns: Printable execution report.

Side Effects: Calls `Game.market.deal`; appends history and updates plan execution fields.

Execution Level: Approval Required

Source: `src/market_console.js:1688`, `src/market_console.js:3121`

`market.execute(planId)`

Description: Alias for `market.executePlan(planId)`.

Parameters: `planId`: saved plan id.

Example: `market.execute("ms_223455_H_W42N9")`

Returns: Printable execution report.

Side Effects: Calls `Game.market.deal`; appends history and updates plan execution fields.

Execution Level: Approval Required

Source: `src/market_console.js:3122`

`market.history(filter)`

Description: Show recent market execution history.

Parameters: `filter`: optional resource, room, or all.

Example: `market.history("H")`

Returns: Printable block.

Side Effects: Trims history to current limit.

Execution Level: Planning

Source: `src/market_console.js:1722`, `src/market_console.js:3123`

`market.historySummary()`

Description: Summarize market execution history counts and accounting.

Parameters: None.

Example: `market.historySummary()`

Returns: Printable block.

Side Effects: Reads history.

Execution Level: Read Only

Source: `src/market_console.js:1769`, `src/market_console.js:3124`

`market.historyAudit()`

Description: Audit history for repeated failures, stale plans, and limit blocks.

Parameters: None.

Example: `market.historyAudit()`

Returns: Printable block.

Side Effects: Reads history.

Execution Level: Read Only

Source: `src/market_console.js:1844`, `src/market_console.js:3125`

`market.clearHistory(mode)`

Description: Remove selected execution history entries.

Parameters: `mode`: failed, blocked, stale, deleted, or all.

Example: `market.clearHistory("failed")`

Returns: Printable line.

Side Effects: Deletes history entries.

Execution Level: Planning

Source: `src/market_console.js:1898`, `src/market_console.js:3126`

`market.setHistoryLimit(limit)`

Description: Set maximum market history size.

Parameters: `limit`: integer at least 10.

Example: `market.setHistoryLimit(100)`

Returns: Printable line.

Side Effects: Updates history limit and trims old entries.

Execution Level: Planning

Source: `src/market_console.js:1932`, `src/market_console.js:3127`

`market.historyLimit()`

Description: Show current history retention limit.

Parameters: None.

Example: `market.historyLimit()`

Returns: Printable line.

Side Effects: Ensures default history limit exists.

Execution Level: Planning

Source: `src/market_console.js:1946, src/market_console.js:3128`

`market.buy(resource, amount, roomName)`

Description: Directly buy a resource from the best usable sell order.

Parameters: `resource, amount, roomName`.

Example: `market.buy("energy", 5000, "W41N8")`

Returns: Printable execution line.

Side Effects: Calls `Game.market.deal`.

Execution Level: Executes Immediately

Source: `src/market_console.js:2769, src/market_console.js:3129`

`market.sell(resource, amount, roomName)`

Description: Directly sell a resource to the best usable buy order.

Parameters: `resource, amount, roomName`.

Example: `market.sell("H", 2580, "W42N9")`

Returns: Printable execution line.

Side Effects: Calls `Game.market.deal`.

Execution Level: Executes Immediately

Source: `src/market_console.js:2881, src/market_console.js:3130`

`market.planBuys()`

Description: Print suggested buy or receive actions.

Parameters: None.

Example: `market.planBuys()`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:2982, src/market_console.js:3132`

`market.planSells()`

Description: Print suggested sell or send actions.

Parameters: None.

Example: `market.planSells()`

Returns: Printable block.

Side Effects: Prints report.

Execution Level: Read Only

Source: `src/market_console.js:3030, src/market_console.js:3133`

4. Room Operations

Economy:

- `ops.room([roomName], "economy")`: room energy, storage, hub/controller container, upgrade rate, spawn queue, and hauler mode.
- `ops.room([roomName], "resources")`: storage energy, terminal energy, terminal power, ghodium, minerals, and terminal balance state.
- `ops.terminalStatus([roomName])`: terminal stock and congestion status.
- `market.stock([roomName])`, `market.needs()`, and `market.surplus()`: economy visibility from the market helper.

Logistics:

- `ops.move(resource, amount, roomName, from, to)`: create storage/terminal movement requests.
- `ops.clearTerminal(...)`, `ops.fillTerminal(...)`, `ops.balanceTerminal(...)`, and `ops.balanceTerminals()`: terminal hygiene and balance workflows.
- `ops.requests(...)`, `ops.cancel(...)`, `market.stage(...)`, `market.unstage(...)`, `market.requests(...)`, and `market.cancel(...)`: logistics request visibility and control.

Construction:

- `ops.room([roomName], "build")`: roadmap status, current/future phase work, storage planning, mineral road needs, and next task.
- `ops.phase(roomName)`: undocumented alias for the build section.

Remotes:

- `ops.reserve(targetRoom, [parentRoom])`, `ops.reserved([parentRoom])`, and `ops.cancelReserve(targetRoom)`: reserved-room lifecycle.
- `ops.expand(targetRoom, [parentRoom])`, `ops.expansions()`, and `ops.cancelExpansion(targetRoom)`: expansion lifecycle.
- `ops.attack(targetRoom, [postAction], [parentRoom], [allies])`, `ops.attacks()`, and `ops.cancelAttack(targetRoom)`: attack plan lifecycle and post-action planning.

Defense:

- `ops.room([roomName], "defense")`: alert state, safe mode, hostiles, threat score, response mode, breach severity, tower status, defender readiness, support, and target focus.
- `ops.log([roomName], [limit])` and `ops.logClear([roomName])`: invasion log inspection and cleanup.

CPU:

- `ops.cpu([roomName])` and `ops.cpuStatus([roomName])`: room CPU report.
- `ops.tickRate([sampleTicks|status|cancel])` and `ops.tickSpeed(...)`: wall-clock tick-speed probe.

Reporting:

- `ops.room([roomName], "overview"|"all")`, `ops.rooms()`, and `ops.empire()`: primary room and empire reports.
- `ops.hud(on|off)`, `ops.reports(on|off)`, and `view(on|off)`: reporting visibility controls.

5. Power Operations

GPL:

- `ops.pcl([roomName])` prints global GPL/PCL status and optional room enablement readiness.

PCL:

- `ops.pcl([roomName])` is the PCL summary entry point.
- `ops.powerCreeps()` lists friendly Power Creeps.

Power Spawn:

- `ops.power()` shows empire Power Spawn status.
- `ops.power("ROOM", "detail")` or `ops.room("ROOM", "power")` shows a room power report.
- `ops.power("ROOM", "process", "on"|"off")`, `ops.power("ROOM", "refill", "on"|"off")`, and reserve forms set room-local Power Spawn policy.

Power Creeps:

- `ops.powerCreep(name, "assign", room)` and `ops.powerCreep(name, "unassign")` manage renewal-assist assignment memory.
- `ops.powerCreep(name, "renewAssist", on|off)` toggles per-creep renewal assist.
- `ops.powerCreep(name, "renewStatus")` reports renewal assist state.
- `ops.powerCreep(name, "position", room)` reports target positioning.
- `ops.powerCreep(name, "move", room, target, "check"|"confirm")` checks or confirms movement preparation.
- `ops.powerCreep(name, "spawn", room, "check"|"confirm")`, `ops.powerCreep(name, "renew", room, "check"|"confirm")`, and `ops.powerCreep(name, "generateOps", "check"|"confirm")` are approval-gated lifecycle/power actions.

Operators:

- `ops.operator(name, [room], "powers")` reports `OPERATE_*` readiness and target visibility.
- `ops.operator(name, room, "operateSpawn", [spawnNameOrId], "check"|"confirm")` checks or confirms `PWR_OPERATE_SPAWN`.
- `ops.operator(name, room, "operateExtension", "check"|"confirm")` checks or confirms `PWR_OPERATE_EXTENSION`.

Ops resource management:

- `ops.ops()` shows empire ops inventory.
- `ops.ops("ROOM")` shows room ops inventory.
- `ops.ops("ROOM", "stage", "storage", "terminal", amount)` stages ops between storage and terminal through logistics requests.

6. Market Operations

Visibility:

- `market.rooms()`, `market.stock([roomName])`, `market.needs()`, and `market.surplus()`.

Internal terminal logistics:

- `market.stage(resource, amount, roomName)`, `market.unstage(resource, amount, roomName)`, `market.requests(...)`, `market.cancel(requestId)`, and `market.send(...)`.

Market scanning:

- `market.buyOptions([resource])` and `market.sellOptions([resource])`.

Market intelligence:

- `market.readiness([roomName|resource])`, `market.opportunities([resource])`, and `market.recommendations()`.

Dry-run planning:

- `market.planSell(...)`, `market.planBuy(...)`, `market.plans(...)`, `market.plan(...)`, `market.planSummary()`, `market.planReview(...)`, `market.planAudit()`, and `market.clearPlan(...)`.
- Deprecated aliases remain implemented: `market.deletePlan(...)`, `market.removePlan(...)`, and `market.clearPlans()`.

Execution design and safety:

- `market.executionStatus()`, `market.executionDryRun(planId)`, `market.executionLimits()`, `market.setExecutionLimit(name, value)`, `market.clearExecutionLimit(name)`, `market.limits()`, `market.setLimit(name, value)`, `market.clearLimit(name)`, `market.executePlan(planId)`, and `market.execute(planId)`.

Execution history:

- `market.history([filter])`, `market.historySummary()`, `market.historyAudit()`, `market.clearHistory(mode)`, `market.setHistoryLimit(limit)`, and `market.historyLimit()`.

Manual trading:

- `market.buy(resource, amount, roomName)` and `market.sell(resource, amount, roomName)` execute immediately through `Game.market.deal`.

7. Observer Operations

No `observer.*` command root is registered in source.

Observer reporting is exposed through room reports:

- `ops.room("observer")`: prints observer report for the current/default room.
- `ops.room("ROOM", "observer")`: prints observer report for a specific room.
- `ops.room("ROOM", "all")`: includes observer lines with the rest of the room report.

The observer section reports whether observer logic is enabled, visible observer count, last target, last result, queued targets, target list, intel count, newest/oldest intel age, run interval, and last run tick. Source: `src/room_reporting.js:1576`.

8. Troubleshooting

Cannot create first Power Creep:

- Run `ops.pcl("ROOM")` to inspect GPL/PCL and room enablement readiness.
- Run `ops.powerEnable("ROOM", "check")` before attempting confirmation.
- If readiness is blocked, inspect the printed next step from the enablement report.

Power Spawn not processing:

- Run `ops.power()` for empire summary.

- Run `ops.power("ROOM", "detail")` or `ops.room("ROOM", "power")`.
- Check global and room-local process/refill overrides, energy/power levels, terminal balance, refill pending count, and blocked reason.
- Use `ops.power("ROOM", "process", "on")` or `ops.power("ROOM", "refill", "on")` only when you intend to change policy.

Remote reservation lost:

- Run `ops.reserved("PARENT")` to check active plans.
- Run `ops.room("PARENT", "defense")` and `ops.log("TARGET")` for threat context.
- Recreate or update with `ops.reserve("TARGET", "PARENT")`.

Room CPU high:

- Run `ops.cpu("ROOM")`.
- Review pressure, trend, scheduler skips, and top section cost lines.
- Run `ops.tickRate("status")` if a wall-clock probe is already active, or `ops.tickRate(5)` to sample.

Terminal clogged or market blocked:

- Run `ops.terminalStatus("ROOM")` and `market.readiness("ROOM")`.
- Use `ops.clearTerminal("ROOM")` for source-selected cleanup or `ops.clearTerminal("ROOM", resource, amount)` for a specific resource.
- Run `market.requests("ROOM", "all")` to find stuck logistics.

Market plan will not execute:

- Run `market.planReview(planId)` and `market.executionDryRun(planId)`.
- Check stale order, terminal cooldown, resource amount, terminal energy, credits, and execution limits.
- Run `market.executionLimits()` before changing limits.

Power Creep operation does not fire:

- Run `ops.operator(name, room, "powers")`.
- Use check mode first for the specific action.
- Use `confirm` only when the check report is ready and the target is correct.

9. Quick Reference

Read-only first:

- Rooms: `ops.room("ROOM")`, `ops.room("ROOM", "economy")`, `ops.room("ROOM", "defense")`, `ops.rooms()`, `ops.empire()`
- CPU: `ops.cpu("ROOM")`, `ops.tickRate(5)`, `ops.tickRate("status")`
- Power: `ops.power()`, `ops.power("ROOM", "detail")`, `ops.pcl("ROOM")`, `ops.powerCreeps()`
- Observer: `ops.room("ROOM", "observer")`
- Market: `market.readiness()`, `market.opportunities("H")`, `market.recommendations()`

Planning and memory updates:

- Reservations: `ops.reserve("TARGET", "PARENT")`, `ops.cancelReserve("TARGET")`
- Expansions: `ops.expand("TARGET", "PARENT")`, `ops.cancelExpansion("TARGET")`
- Attack plans: `ops.attack("TARGET", "expand", "PARENT", ["ALLY"])`, `ops.cancelAttack("TARGET")`
- Logistics: `ops.move("H", 50000, "ROOM", "terminal", "storage")`, `ops.fillTerminal("ROOM", "energy", 10000)`, `ops.clearTerminal("ROOM")`
- Market plans: `market.planSell("H", 10000, "ROOM")`, `market.planReview(planId)`, `market.executionDryRun(planId)`

Executing commands:

- Power: `ops.powerCreep(name, "generateOps", "confirm")`, `ops.operator(name, "ROOM", "operateSpawn", "Spawn1", "confirm")`, `ops.powerEnable("ROOM", "confirm", name)`
- Market: `market.executePlan(planId)`, `market.buy(resource, amount, roomName)`, `market.sell(resource, amount, roomName)`, `market.send(resource, amount, fromRoom, toRoom)`

Command Surface Findings

Counts by category:

- Global callable aliases: 1 (view).
- Global convenience constants: 2 (on, off).
- `ops.*` callable functions: 37.
- `market.*` callable functions: 51.
- Total callable operator commands documented: 89.
- Ops help signature rows: 40.
- Market help signature rows: 65.

Missing documentation:

- `ops.tickSpeed(sampleTicks)` is implemented as an alias for `ops.tickRate` but has no ops help entry.
- `ops.cpuStatus(roomName)` is implemented as an alias for `ops.room(roomName, "cpu")` but has no ops help entry.
- `ops.phase(roomName)` is implemented as an alias for `ops.room(roomName, "build")` but has no ops help entry.
- `market.version` and `market.config` are exposed fields, not commands; they are intentionally excluded from the command count.

Duplicate commands:

- `ops.tickSpeed` duplicates `ops.tickRate`.
- `ops.cpuStatus` duplicates `ops.cpu`.
- `ops.phase` duplicates `ops.room(..., "build")`.
- `market.limits`, `market.setLimit`, and `market.clearLimit` duplicate the execution-limit commands.
- `market.execute` duplicates `market.executePlan`.
- `market.deletePlan`, `market.removePlan`, and `market.clearPlans` are deprecated aliases for `market.clearPlan`.

Stale help entries:

- No `ops.help()` or `market.help()` entry was found that references a missing callable implementation.
- Some help rows intentionally document variants of the same function, for example `market.stock()` and `market.stock(roomName)`.

Broken references:

- No broken help-to-implementation references were found in the source-registered help surfaces.
- No `attack.*`, `expansion.*`, `reservation.*`, `logistics.*`, `power.*`, `powerCreep.*`, `operator.*`, or `observer.*` global roots are registered. These workflows are exposed under `ops.*`.

Opportunities to consolidate command surfaces:

- Add help rows for implemented aliases or intentionally hide them as internal compatibility aliases.
- Consider documenting `ops.room` sections directly in `ops.help()` so `observer`, `resources`, and `advanced` are discoverable from the console.
- Consider aligning deprecated market plan aliases around one compatibility section so operators do not confuse aliases with preferred commands.
- Consider separating immediate market execution commands from approval-gated saved-plan execution in `market.help()` to make risk levels clearer.